

Metody Jarratta i Halley'a

Wyznaczanie zer wielomianu w dziedzinie zespolonej i wizualizacja zbieżności metod

Błażej Klepacki
Jan Kochaniak

Cel projektu

- Numeryczne **wyznaczanie zer wielomianów** w dziedzinie zespolonej.
- **Porównanie** wydajności, stabilności i kosztu obliczeniowego dwóch różnych podejść iteracyjnych.
- **Wizualizacja** i badanie fraktalnej natury zbieżności.

Historia metod

Edmond Halley (1694 r.): Angielski astronom (znany z komety Halley'a). Udoskonalił metodę Newtona używając paraboli zamiast linii prostej. Klasyczne podejście oparte na badaniu "krzywizny" (II pochodna).

Peter Jarratt (1966 r.): XX-wieczny analityk numeryczny. Zamiast liczyć uciążliwą II pochodną, jego metoda sprytnie "próbkuje" spadek funkcji w dwóch miejscach naraz.

Metoda Halleya

$$z_{k+1} = z_k - \frac{2f(z_k)f'(z_k)}{2(f'(z_k))^2 - f(z_k)f''(z_k)}.$$

Rząd zbieżności: $p = 3$ (sześcienna dla pierwiastków jednokrotnych)

Koszt obliczeniowy: wymaga 3 ewaluacji w jednym kroku (f , f' , f'')

Implementacja: opiera się na schemacie Hornera dla wielomianów i ich pochodnych

Metoda Jarratta

$$y_k = z_k - \frac{2}{3} \cdot \frac{w(z_k)}{w'(z_k)}.$$

$$z_{k+1} = z_k - \frac{w(z_k)}{w'(z_k)} \cdot \frac{3w'(y_k) + w'(z_k)}{6w'(y_k) - 2w'(z_k)}.$$

Rząd zbieżności: $p = 4$

Koszt obliczeniowy: wymaga 3 ewaluacji w jednym kroku ($f(z_n)$, $f'(z_n)$, $f'(y_n)$)

Wyższy rząd przy koszcie podobnym do Halleya (brak II pochodnej)

Implementacja - kluczowe elementy i najważniejsze funkcje

Horner

Zoptymalizowany do metody: dla Halleya algorytm zwraca od razu pierwszą i drugą pochodną, dla Jarratta zatrzymuje się na pierwszej.

Wersja zwektoryzowana i niezwektoryzowana.

Halley

Podejście globalne (macierzowe): skrypt operuje od razu na gęstej siatce płaszczyzny zespolonej pod kątem wizualizacji fraktali.

Inteligentne maskowanie logiczne: algorytm w każdej iteracji wyklucza z obliczeń te piksele, które już zbiegły do zera, co drastycznie oszczędza zasoby.

Architektura monolityczna: cała mechanika iteracyjna i mapowanie zer zamknięte w jednej funkcji.

Jarratt

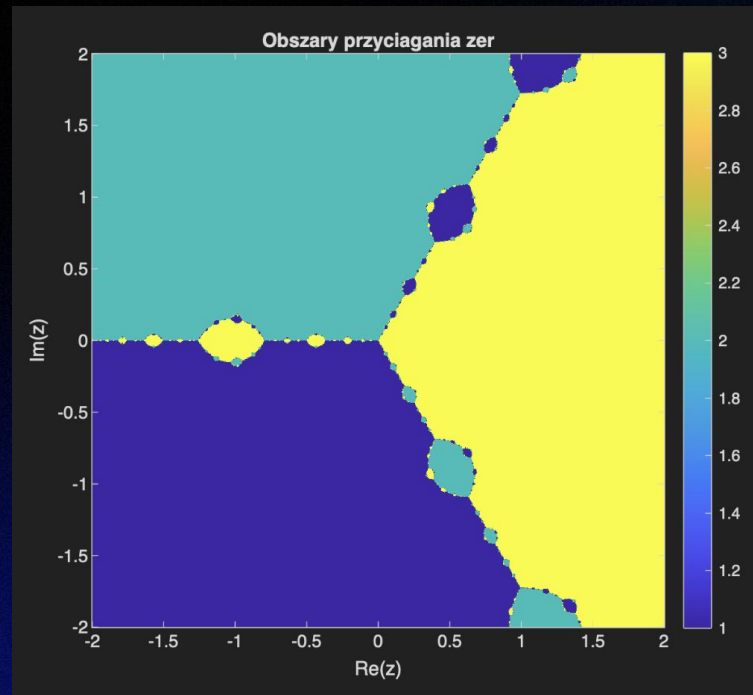
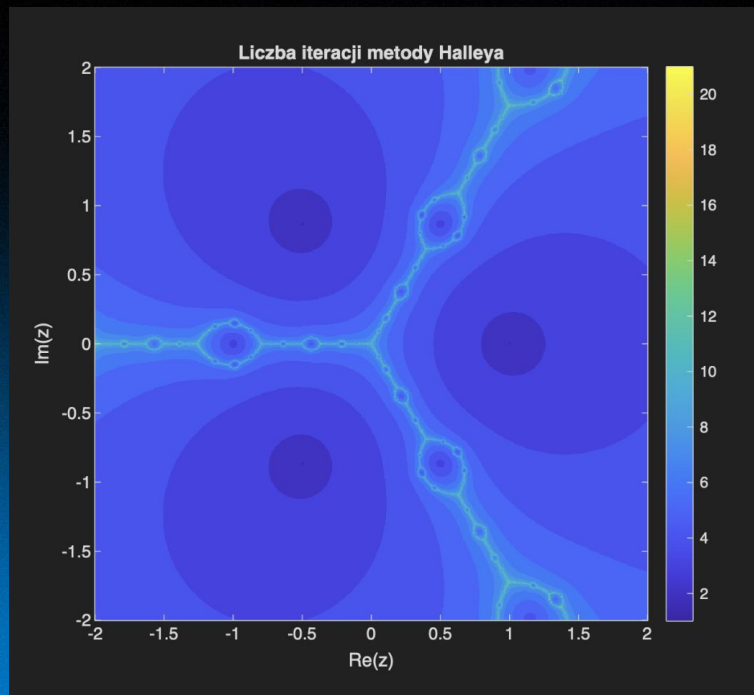
Architektura modułowa: rozbita na niezależne klocki: pojedynczy krok, główną pętlę iteracyjną oraz nadrzędną funkcję grupującą.

Wbudowane bezpieczniki: funkcja kroku iteracyjnego na bieżąco wyłapuje osobliwości i chroni program przed „wybuchem” (dzieleniem przez zero).

Optymalizacja szukania: zamiast przeszukiwać w ciemno, skrypt wypuszcza punkty startowe z okręgów i sprytnie klasteryzuje unikalne wyniki.

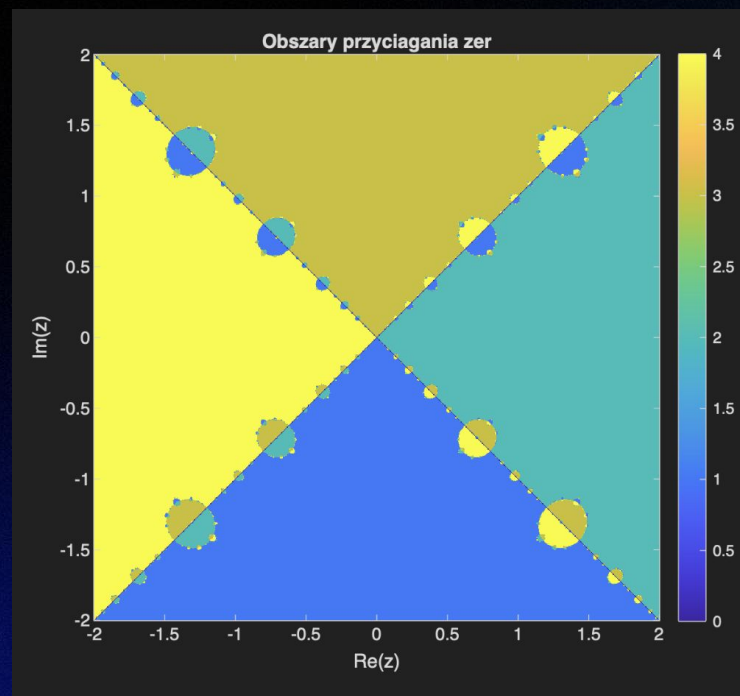
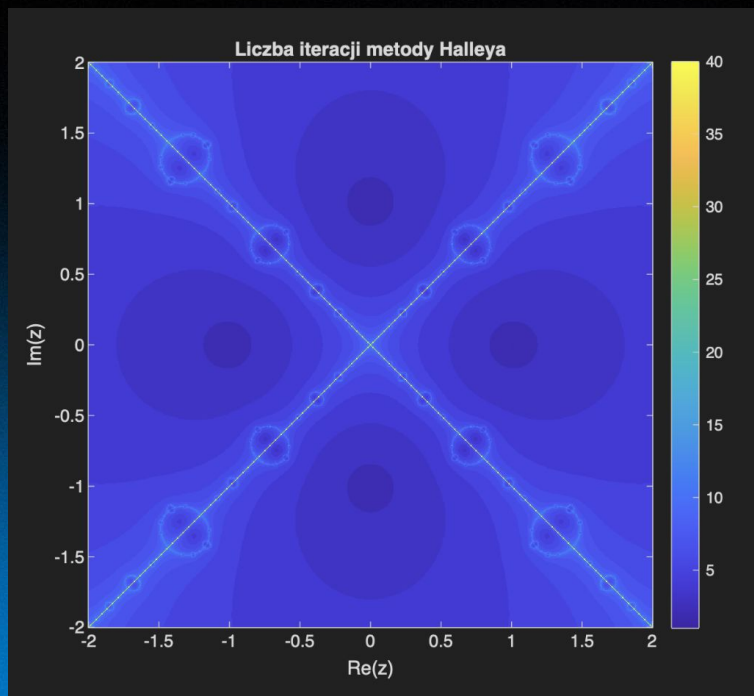
Fraktale Halley'a - baseny przyciągania

$$w(z) = z^3 - 1$$



Fraktale Halley'a - baseny przyciągania

$$w(z) = z^4 - 1$$



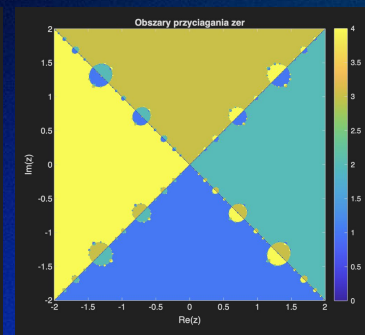
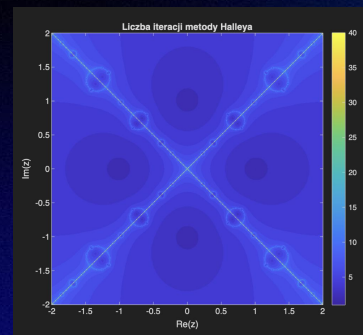
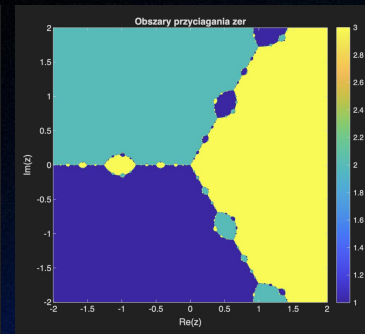
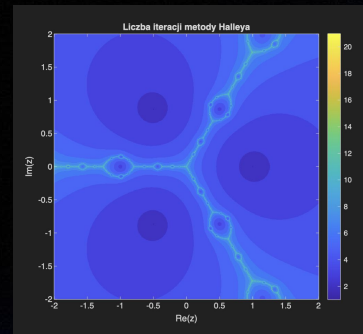
Fraktale Halley'a - baseny przyciągania

analiza / wnioski

Anatomia zbieżności: w ciemnoniebieskich obszarach metoda Halley'a w pełni wykorzystuje swój trzeci rząd zbieżności i zbiega w kilka kroków. Jasne linie idealnie pokrywają się z granicami basenów przyciągania. To strefy "chaosu", gdzie algorytm błądzi i potrzebuje najwięcej iteracji do znalezienia drogi do zera.

Efekt motyla w dziedzinie zespolonej: niestabilność na granicach - wynik algorytmu zależy drastycznie od wyboru punktu startowego z_0 . Minimalne przesunięcie punktu startowego to, do którego pierwiastka zbiegnie funkcja.

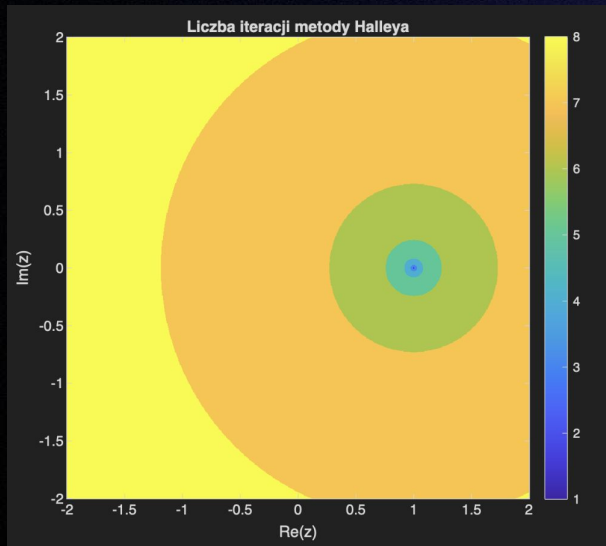
Symetria wielomianu: struktura fraktala bezbłędnie oddaje naturę badanej funkcji - widoczna jest idealna 3-krotna symetria dla z^3-1 oraz 4-krotna dla z^4-1 .



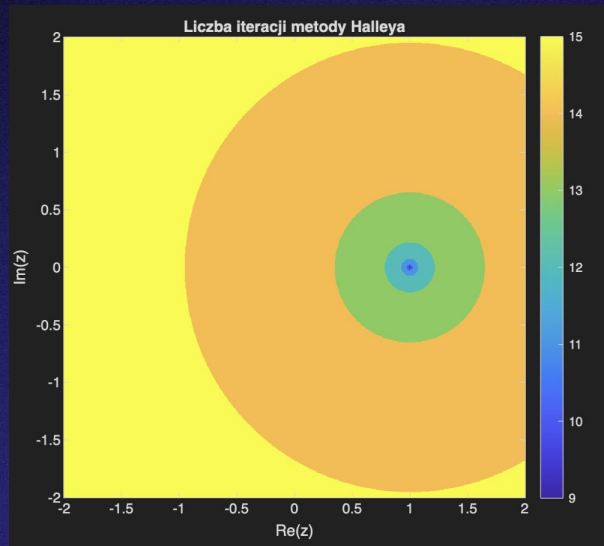
Metoda Halley'a - testy parametryczne i brzegowe

Wpływ tolerancji na liczbę iteracji na przykładzie $w(z) = (z-1)^2$ oraz test pierwiastków wielokrotnych
Metoda Halleya traci lokalną zbieżność 3. rzędu na rzecz wolnej zbieżności liniowej.

tolerancja 10^6



tolerancja 10^{14}



Metoda Halley'a - testy parametryczne i brzegowe

Wpływ rozmiaru siatki

Test: Zwiększanie rozdzielczości generowanego fraktala (z siatki 800x800 na 2000x2000 punktów).

Wniosek: Dzięki zaimplementowaniu maskowania skrypt na bieżąco usuwa z pamięci punkty, które już zbiegły, skupiając moc obliczeniową wyłącznie na trudnych rejonach granicznych. Dzięki temu czas obliczeń rośnie wolniej, niż wynikałoby to ze wzrostu liczby punktów.

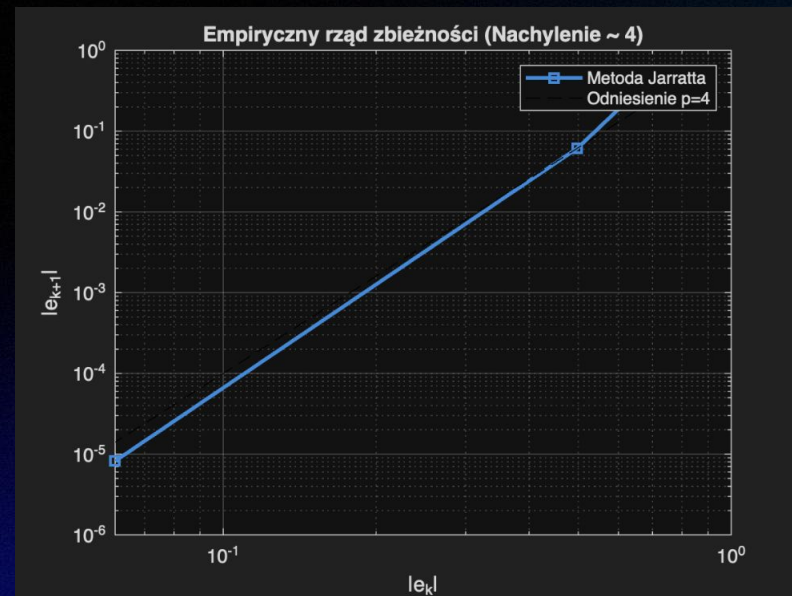
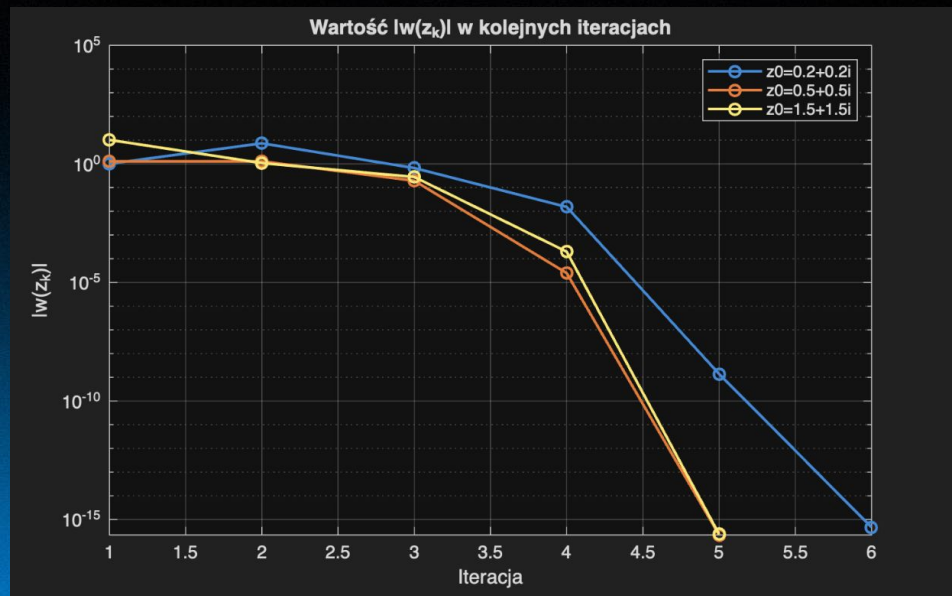
Osobliwości i brak rozwiązania

Test: Punkt startowy powodujący wyzerowanie mianownika we wzorze Halley'a.

Wniosek: Dzięki warunkowi $\text{abs}(\text{mianownik}) > 1^{-12}$ w kodzie, algorytm nie przerywa pracy z błędem w przypadku trafienia w lokalne ekstremum. Takie punkty "zamrażają" się i są widoczne na fraktalach jako czarne/puste piksele (obce punkty stałe), co chroni główną pętlę przed awarią.

Metoda Jarratta

Wizualizacja zbieżności dla $w(z) = z^3 - 1$

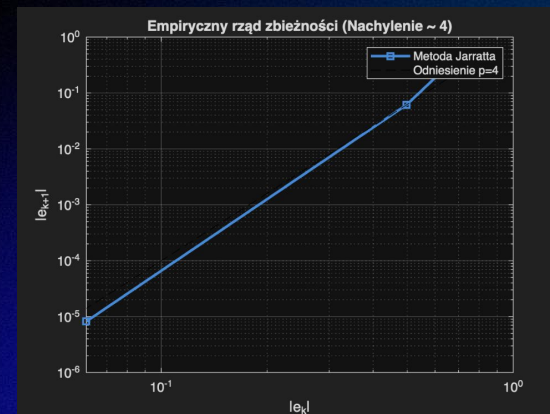
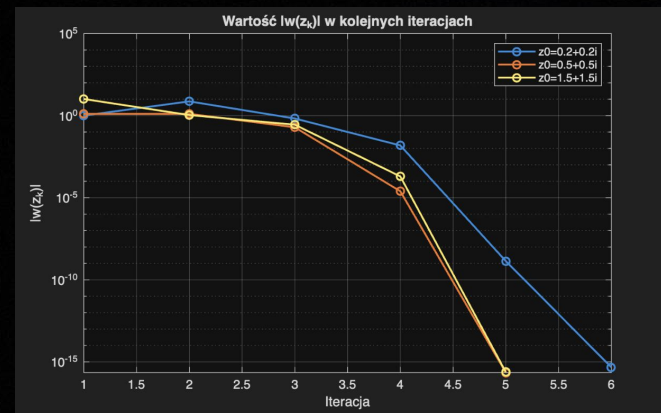


Metoda Jarratta

Wizualizacja zbieżności dla $w(z) = z^3 - 1$ - analiza i wnioski

Błyskawiczny spadek błędu: Oś Y ma skalę logarytmiczną. Wykres wyraźnie pokazuje, że metoda na początku "szuka" drogi, ale gdy tylko trafi w pobliże zera (okolice 3. iteracji), błąd pikuje pionowo w dół. Osiągnięcie precyzji rzędu 10^{-15} zajmuje zaledwie ułamek sekundy.

Empiryczny dowód poprawności: To najważniejszy wykres potwierdzający działanie naszego kodu. Zestawiliśmy tu błąd z bieżącej iteracji z błędem w iteracji następnej. Nachylenie uzyskanej prostej pokrywa się idealnie z funkcją odniesienia, co dowodzi matematycznie, że nasza implementacja w 100% zachowuje lokalną, czwartorzędowną zbieżność ($p=4$)



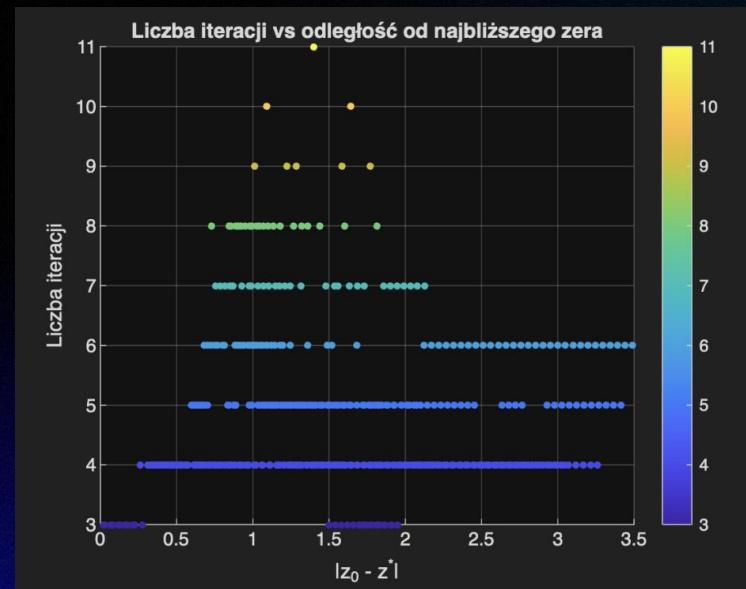
Metoda Jarratta - wizualizacja wyników

Wrażliwość na punkt startowy - analiza kosztu obliczeniowego w zależności od odległości

Zależność od parametru wejściowego: wykres obrazuje, ile iteracji musiał wykonać algorytm w zależności od tego, jak daleko leżał punkt startowy z_0 od znalezionej ostatecznie zera z^* .

Strefa pewności: dla punktów startowych leżących blisko zera (odległość poniżej 0.5), algorytm zawsze zamyka się w stałych 4 iteracjach.

Błądzenie w chaosie: Wraz ze wzrostem odległości (powyżej 1.0) pojawia się ogromny rozrzut wyników. Liczba iteracji skacze punktowo nawet do 11. Udowadnia to, że "odległość" nie jest jedynym czynnikiem - kluczowe jest to, czy daleki punkt startowy niefortunnie wylądował na "fraktalnej granicy", co zmusza algorytm Jarratta do długiego szukania właściwego obszaru zbieżności.



Metoda Jarratta - testy brzegowe i pułapki

Osobliwość w punkcie startowym

Problem: Dla równania $w(z)=z^4-1$ przy starcie z $z_0=0$, pochodna znika ($w'(0)=0$), co prowadzi do dzielenia przez zero we wzorze.

Rozwiązanie w kodzie: Algorytm nie zwraca błędu NaN. Wbudowany w funkcję bezpiecznik ($\text{abs}(\text{dwz}) < 1e-14$) przerywa iterację ze statusem osobliwość.

Wniosek: Nawet minimalna perturbacja punktu startowego (np. przesunięcie na 0.01i) ratuje sytuację i algorytm zbiega w 5 iteracjach.

Pułapka osi rzeczywistej

Problem: Szukając zespolonych pierwiastków $\pm i$ dla wielomianu $w(z)=z^2+1$ wystartowaliśmy z osi rzeczywistej ($z_0=2.0$).

Wynik: Skrypt utknął w pętli i osiągnął limit iteracji.

Wniosek: Operacje arytmetyczne metody Jarratta na liczbach rzeczywistych nigdy nie wyjdą poza dziedzinę rzeczywistą. Narzuca to konieczność inicjowania algorytmu z niezerową częścią urojoną.

Metoda Jarratta - wydajność i testy parametryczne

Bezpośrednie starcie: Jarratt vs Newton

Test: Analiza liczby iteracji dla $w(z)=z^3-1$ przy 5 różnych punktach startowych.

Wynik: Jarratt za każdym razem potrzebował dokładnie połowę mniej iteracji niż klasyczna metoda Newtona.

Wniosek: Pojedynczy krok Jarratta jest nieco droższy (3 ewaluacje Hornera zamiast 2), jednak dzięki wyższemu rzędowi zbieżności, całkowity czas obliczeń wynosi zaledwie ok. 75% czasu Newtona.

Odporność na rygorystyczną tolerancję (ϵ)

Wniosek z testów: Dzięki zbieżności 4. rzędu ($p=4$), wymuszenie tolerancji na poziomie zera maszynowego (10^{-14}) nie obciąża programu. Ostatnia iteracja potrafi dorzucić kilkanaście poprawnych cyfr po przecinku w jednym kroku.

Wpływ wyboru z_0 – Klasteryzacja (Rozwiązanie globalne)

Wniosek: Aby uodpornić program na błędzenie z pojedynczego punktu, zaimplementowaliśmy nadrzędną funkcję. Generuje ona automatycznie dziesiątki punktów na okręgach o promieniach 0.5, 1.0, 1.5 i 2.5, a następnie sprytnie filtruje unikalne zera. Eliminuje to wadę lokalności algorytmu.

**Dziękujemy za
uwagę**